

Reinforcement Learning for Algorithmic Trading – WiDS Final Report

Sahil M Deo

Winter in Data Science (WiDS) Program

Abstract

This report presents the work completed as part of the Winter in Data Science (WiDS) program. The project focuses on reinforcement learning methods, starting from multi-armed bandits and Markov Decision Processes to implementing a Q-Learning based trading agent. The final system models stock trading as a sequential decision making problem and evaluates the learned policy against a traditional buy-and-hold strategy.

Introduction

Reinforcement Learning (RL) provides a mathematical framework for learning optimal decision making strategies through interaction with an environment. Financial markets present an interesting yet challenging application domain due to uncertainty, noise, and non-stationarity. This project explores how classical RL techniques can be applied to simplified trading environments to gain practical understanding of the underlying theory.

The project was developed over four weeks, beginning with Python tools and bandit algorithms and culminating in a full Q-Learning based trading agent.

Tools and Technologies

The project was implemented primarily in Python using NumPy for numerical computation, Pandas for handling financial time-series data, and Matplotlib for visualization. GitHub was used for version control and collaboration. The reinforcement learning concepts were studied using Sutton and Barto's textbook as the main theoretical reference.

Multi-Armed Bandit Methods

Three action-selection strategies were implemented: Greedy, Epsilon-Greedy, and Upper Confidence Bound (UCB). These methods highlight the exploration–exploitation trade-off. Empirical simulations demonstrated how purely greedy strategies may converge to suboptimal arms, while UCB and epsilon-greedy balance information gathering with reward maximization.

Markov Decision Process Modeling

Several environments including Bandit Walk, Slippery Walk, and Frozen Lake were formulated as Markov Decision Processes. Each environment was defined using a tuple of states, actions, transition probabilities, rewards, and discount factor. Python utilities were written to automatically construct transition matrices and reward tables, enabling scalable experimentation.

Financial Data Analysis

Historical stock price data was analyzed using Pandas. Market indicators such as Open, Close, and Adjusted Close prices were studied. A simple moving average crossover strategy was implemented to understand baseline algorithmic trading behavior before introducing reinforcement learning methods.

Q-Learning Theory

Q-Learning is a model-free temporal difference learning algorithm that estimates the optimal state-action value function. The update rule is derived from the Bellman optimality equation and allows learning directly from interaction without requiring transition probabilities. Convergence is guaranteed under sufficient exploration and appropriate learning rates.

Trading Environment Design

A custom trading environment was designed with actions {buy, sell, hold}. The state representation was derived from time indices and price information. The reward was defined as the total portfolio value including held shares. Episodes terminated at the end of the historical data sequence. Initial capital was fixed at 100,000 units.

Training and Evaluation

The agent was trained over multiple episodes using an epsilon-greedy exploration strategy. The learned policy was evaluated by tracking portfolio value over time and final profit. A buy-and-hold baseline was implemented for comparison. While the RL agent demonstrated adaptive behavior, results also highlighted limitations of tabular methods in noisy environments.

Discussion and Limitations

The project demonstrates that reinforcement learning can be applied to financial decision making in simplified settings. However, real markets require handling continuous states, transaction costs, risk management, and generalization. Tabular Q-Learning does not scale effectively to large state spaces, motivating the use of function approximation methods such as Deep Q-Networks.

Conclusion

This work provided hands-on experience with reinforcement learning theory and practice. By progressing from bandit problems to a complete trading agent, the project connects mathematical foundations with real-world inspired applications. The framework developed can be extended to more sophisticated market models and deep reinforcement learning approaches.